LM-IDNet: Crucial Prototype Implementation Roadmap

Minimum Ordered Path to a Working IoT Anomaly Detector

Master's Thesis Engineering Plan

July 23, 2026

Abstract

This document is the minimum ordered implementation path extracted from the Atomic Implementation Roadmap. It ends with a working prototype that can deterministically ingest data, produce validated time windows, fit a Dirichlet–Multinomial model, evaluate likelihoods using the LM backend, calibrate a threshold, and emit anomaly events. The original task identifiers are retained for traceability. Tasks omitted from this document are additional validation, comparative research, adaptation, forecasting, deployment, or polishing work; they remain important for the final thesis artifact but do not block the first working prototype.

Contents

1 How to Use This Roadmap

Implement tasks strictly in document order. A task is Done only when its stated verification passes and no previously completed verification regresses. Keep generated evidence under `reports/` or `artifacts/`. Do not begin the next phase until its exit gate passes.

2 Stage 1: Executable Project Foundation

2.1 F0.02: Adopt a source-layout package

Implementation. Define one import strategy for `src/`; add package metadata in `pyproject.toml`; expose a console entry point such as `lm-idnet`; remove runtime `sys.path` mutation after migration.

Verification and acceptance gate. Install the project into a fresh virtual environment in editable mode, execute `python -c "import ..."`, invoke `lm-idnet -help`, and run tests without setting `PYTHONPATH`.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.2 F0.04: Centralize typed configuration

Implementation. Define a Pydantic configuration model for paths, dates, categories, seeds, estimator settings, calibration, and output locations. Reject unknown fields and invalid cross-field combinations.

Verification and acceptance gate. Unit-test missing fields, extra fields, invalid dates, duplicate categories, non-positive window sizes, invalid quantiles, and a known-valid configuration. Snapshot the normalized configuration.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.3 F0.09: Define the testing architecture

Implementation. Configure pytest markers for `unit`, `integration`, `numerical`, and `slow`. Establish fixture factories for count matrices, windows, configurations, and temporary artifact stores.

Verification and acceptance gate. Run each marker independently; ensure no test is silently uncollected and that the default fast suite excludes only explicitly marked slow or data-heavy tests.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.4 F0.17: Define exception and failure semantics

Implementation. Create domain exceptions for configuration, ingestion, validation, numerical precision, convergence, artifact compatibility, and policy rejection. Ensure commands fail closed when model integrity is uncertain.

Verification and acceptance gate. Unit-test every exception mapping to CLI exit code and message. Corrupt a model checksum and verify scoring stops rather than continuing with a warning.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.5 F0.18: Version schemas and artifacts

Implementation. Assign semantic schema versions to processed datasets, models, thresholds, events, and experiment manifests. Reject unknown major versions.

Verification and acceptance gate. Round-trip each schema through JSON and reject an unsupported future major version.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.6 F0.06: Standardize the command-line interface

Implementation. Implement the prototype subcommands `preprocess`, `train`, `calibrate`, and `score`. Make errors non-zero and machine-readable where appropriate.

Verification and acceptance gate. For every implemented subcommand, test `-help`, missing arguments, invalid configuration, and a minimal success case. Assert exit codes and essential output fields.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: The package installs cleanly; typed configuration, tests, versioned schemas, failure semantics, and the four prototype commands work from a fresh environment.

3 Stage 2: Reproducible Data Setup

3.1 P0.01: Inventory source captures

Implementation. Enumerate expected and present captures with date, byte size, packet count where readable, checksum, and parser status. Record missing or duplicate files without silently skipping them.

Verification and acceptance gate. Compare inventory entries with the filesystem; assert unique checksums unless duplication is explained; fail when a configured required capture is missing.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.2 P0.02: Verify capture integrity

Implementation. Use a PCAP validation tool or complete streaming parse to detect truncation, invalid timestamps, and corrupt records. Record recoverable warnings separately from fatal corruption.

Verification and acceptance gate. Parse every configured source to end-of-file. Store counts and warnings; rerun and confirm identical counts. Quarantine a deliberately truncated fixture and verify the failure classification.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.3 P0.05: Freeze temporal partitions

Implementation. Create disjoint fit, calibration, development-test, and final-test date lists. Preserve chronology and prohibit random window mixing across partitions.

Verification and acceptance gate. Automated tests assert non-overlap, chronological order, capture-level grouping, and that final-test identifiers are unavailable to training and threshold code.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.4 P0.06: Create dataset fingerprints

Implementation. Hash the ordered file list, individual inputs, feature taxonomy, window policy, and partition definition into a dataset version identifier.

Verification and acceptance gate. Changing any file, category order, window length, or date assignment must change the fingerprint; repeating unchanged inputs must reproduce it.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.5 P0.07: Centralize deterministic seeds

Implementation. Define named seeds for simulation and model fitting. Propagate generators explicitly instead of using global state.

Verification and acceptance gate. Run the smoke experiment twice in separate processes and compare model parameters, injected anomalies, metrics, and normalized reports exactly or within a declared numerical tolerance.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.6 P0.10: Build a tiny public smoke fixture

Implementation. Create a payload-free synthetic packet/count fixture small enough for tests, with known timestamps, categories, silent windows, and expected outputs.

Verification and acceptance gate. Verify the fixture contains no real identifiers, its license permits distribution, and its expected preprocessing result is asserted exactly in integration tests.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: Inputs are inventoried and validated, chronological partitions and the dataset fingerprint are frozen, and a safe deterministic smoke fixture exists.

4 Stage 3: Canonical Windowed Data

4.1 P1.01: Define canonical category semantics

Implementation. Choose lowercase canonical names and explicitly decide whether SSDP is exclusive of UDP or counted in both. Record category order and classification priority.

Verification and acceptance gate. Use packets representing TCP, ordinary UDP, SSDP/UDP:1900, ARP, and unknown protocols; assert exactly the required counts and fixed column order.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.2 P1.03: Harden timestamp parsing

Implementation. Convert all packet times to timezone-aware UTC, reject non-finite values, and preserve sufficient precision. Record original capture time-zone assumptions.

Verification and acceptance gate. Test epoch boundaries, fractional seconds, naive values, out-of-order packets, and daylight-saving examples; assert normalized UTC values.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.3 P1.04: Specify window boundaries

Implementation. Implement half-open windows $[t, t + \Delta)$, with an explicitly chosen origin and deterministic boundary behavior.

Verification and acceptance gate. Place packets immediately before, on, and immediately after a boundary and assert assignment to exactly one correct window for every supported window length.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.4 P1.05: Preserve window timestamps

Implementation. Define each window record with device ID, start/end UTC timestamps, ordered counts, total count, missingness state, and metadata.

Verification and acceptance gate. Schema round-trip preserves timestamps and category order; assert $N_t = \sum_{k=1}^{K} x_{tk}$ for every record.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.5 P1.06: Represent silent windows

Implementation. Materialize elapsed capture windows with zero total count and label them `observed-silent`; do not confuse them with absent capture coverage.

Verification and acceptance gate. Use a fixture with a deliberate quiet interval and verify all expected zero windows exist, remain ordered, and survive JSON/matrix export.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.6 P1.07: Represent missing coverage

Implementation. Define a separate `missing` state for gaps where observation is unavailable; prohibit automatic conversion of missing coverage to zeros.

Verification and acceptance gate. Use a fixture with declared capture discontinuity and assert missing windows are excluded or handled according to policy, never counted as observed silence.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.7 P1.08: Validate count values

Implementation. Reject booleans, negatives, fractional values, NaN, infinity, and integer overflow; use a documented integer width.

Verification and acceptance gate. Parameterized tests exercise every invalid type and maximum supported values. Valid arrays retain exact integer values through all exports.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.8 P1.09: Validate temporal streams

Implementation. Check monotonicity, duplicate windows, overlap, expected duration, device continuity, and category-schema consistency.

Verification and acceptance gate. Inject one defect of each type and assert a specific validation error containing dataset and window identifiers. A valid multi-day stream passes.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.9 P1.10: Make PCAP parsing resource-safe

Implementation. Stream packets with bounded memory, close readers on success and failure, constrain pathological packet handling, and count unsupported packets.

Verification and acceptance gate. Measure memory over a representative capture, inject parser exceptions, and assert file descriptors close and partial outputs are not promoted.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.10 P1.12: Export canonical JSON

Implementation. Define canonical JSON with schema version, dataset fingerprint, metadata, window records, and checksums.

Verification and acceptance gate. Round-trip a multi-day fixture and compare every field. Verify stable serialization and checksum across repeated exports.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.11 P1.13: Export modeling matrices

Implementation. Export timestamps, missingness mask, category columns, totals, device ID, and partition without losing provenance.

Verification and acceptance gate. Load canonical JSON and matrix output and assert row count, order, counts, timestamps, and metadata agree exactly.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.12 P1.14: Load matrices by partition

Implementation. Implement a loader that selects only declared partition identifiers and returns $X \in \mathbb{N}^{R \times K}$ plus aligned metadata.

Verification and acceptance gate. Assert shape, dtype, non-negativity, category order, and exact selected dates. Requesting an undeclared partition must fail.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: The smoke and configured captures produce schema-valid, time-indexed, checksum-backed matrices with correct silence, missingness, and partition semantics.

5 Stage 4: Dirichlet–Multinomial Model

5.1 P2.01: Define estimator input contract

Implementation. Accept only two-dimensional non-negative integer matrices with $R > 1$, $K > 1$, stable category order, and a documented policy for zero-total rows and all-zero categories.

Verification and acceptance gate. Parameterized tests reject every invalid dimension and policy violation; a valid matrix is not mutated by training.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.2 P2.02: Verify the likelihood formula

Implementation. Implement the reference log probability

$$\log p(\mathbf{x} \mid \boldsymbol{\alpha}) = \log \Gamma(N+1) - \sum_{k=1}^{K} \log \Gamma(x_k + 1) \tag{1}$$

$$+ \log \Gamma(\alpha_0) - \log \Gamma(N + \alpha_0) + \sum_{k=1}^{K} \left[\log \Gamma(x_k + \alpha_k) - \log \Gamma(\alpha_k)\right], \tag{2}$$

where $\alpha_0 = \sum_k \alpha_k$. Make inclusion of the multinomial coefficient explicit.

Verification and acceptance gate. Compare small cases with direct arbitrary-precision probability enumeration and verify the coefficient convention with a hand-calculated example.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.3 P2.03: Implement robust initialization

Implementation. Provide mean-concentration and moment-based initializers, floors for sparse categories, and deterministic fallback behavior.

Verification and acceptance gate. Test balanced, highly skewed, sparse, and nearly constant synthetic matrices; every returned α_k must be finite and strictly positive.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.4 P2.04: Harden the fixed-point update

Implementation. Separate one Minka update from the iteration loop, validate numerator and denominator, guard positivity, and detect non-finite or zero updates.

Verification and acceptance gate. Unit-test the update against a stored high-precision example and force each guard branch with synthetic pathological inputs.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.5 P2.05: Define convergence criteria

Implementation. Require both parameter stability and likelihood stability, use absolute plus relative tolerances, impose maximum iterations, and distinguish converged, stalled, diverged, and exhausted outcomes.

Verification and acceptance gate. Construct fixtures for each outcome and assert status, iteration count, and diagnostic reason. Never label maximum-iteration exhaustion as convergence.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.6 P2.07: Activate the modeling stage

Implementation. Load only the fit partition, initialize and fit the estimator, derive α_0, $p_k = \alpha_k/\alpha_0$, and $\psi = 1/\alpha_0$, then persist diagnostics.

Verification and acceptance gate. Run the CLI on the smoke matrix; assert a valid model artifact, successful exit, correct categories, positive alpha, $\sum_k p_k = 1$, and no access to calibration/test partitions.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.7 P2.08: Define the model artifact

Implementation. Include schema version, model version, categories, parameters, training range, dataset/config/code hashes, backend, convergence, runtime, and integrity checksum.

Verification and acceptance gate. Validate against schema, round-trip exactly, reject category reordering and checksum corruption, and verify no raw packet data is embedded.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.8 P2.09: Recover synthetic parameters

Implementation. Sample multiple seeded datasets from known $\boldsymbol{\alpha}^*$, fit them, and quantify error in mean composition and concentration.

Verification and acceptance gate. Across increasing sample sizes, demonstrate decreasing median estimation error or explain deviations; enforce predeclared tolerances on a stable medium-size fixture.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: Partition-safe training produces an integrity-checked converged model and recovers known synthetic parameters within declared tolerances.

6 Stage 5: LM Likelihood Backend

6.1 P3.01: Define the backend protocol

Implementation. Create a stateless `LikelihoodBackend` interface returning value, precision status, error bound when available, backend version, and timing metadata; support single-window and batch calls.

Verification and acceptance gate. Contract-test a fake backend and every real backend for shape, dtype, metadata, immutability, error semantics, and consistent coefficient convention.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.2 P3.02: Implement the standard gammaln backend

Implementation. Move SciPy-based likelihood evaluation behind the interface and support stable vectorized batch evaluation.

Verification and acceptance gate. Compare with the P2 arbitrary-precision oracle across small random cases and with regression fixtures.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.3 P3.03: Build a high-precision oracle

Implementation. Use `mpmath` at configurable precision to evaluate selected log probabilities and paired log-gamma differences outside production code.

Verification and acceptance gate. Validate oracle precision by recomputing at doubled digits; accepted reference digits must remain unchanged. Store reference vectors and provenance.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.4 P3.05: Implement paired log-gamma differences

Implementation. Implement the LM primitive for expressions such as $\log\Gamma(x+a)-\log\Gamma(a)$, with no silent fallback and explicit error estimates.

Verification and acceptance gate. Compare a logarithmic grid of small/large x, small/large a, and low-overdispersion cases with the doubled-precision oracle. Enforce requested error bounds.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.5 P3.06: Assemble the Python LM backend

Implementation. Use the LM primitive for all paired terms in the Dirichlet–Multinomial likelihood and retain an exact/reference path for factorial terms.

Verification and acceptance gate. Contract-test against standard and oracle backends over randomized and adversarial grids. Assert correct precision status propagation.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.6 P3.08: Test extreme numerical regimes

Implementation. Cover zero counts, huge counts, highly imbalanced categories, α_k near the lower bound, very large concentration, and cancellation-prone low-overdispersion cases.

Verification and acceptance gate. No accepted result may be NaN or infinite. Compare to the oracle and enforce tolerances appropriate to each regime.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.7 P3.10: Implement backend selection

Implementation. Select backend only through validated configuration and record backend name/version in every model, score, and manifest.

Verification and acceptance gate. Run identical scoring with each enabled backend and verify artifact metadata. Invalid or unavailable selections must fail before data processing.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: The standard and LM backends satisfy one interface, agree with the independent oracle within declared tolerances, and expose precision failures.

7 Stage 6: Calibration and Anomaly Output

7.1 P4.01: Enforce fit/calibration isolation

Implementation. Load calibration windows through a partition-scoped API that cannot return fit or final-test rows.

Verification and acceptance gate. Assert disjoint identifiers and monkeypatch the fit loader to fail if calibration invokes it. Store partition fingerprints in the threshold artifact.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.2 P4.02: Compute raw window scores

Implementation. Return one log probability $s_t = \log p(\mathbf{x}_t \mid \hat{\boldsymbol{\alpha}})$ and precision metadata per eligible window.

Verification and acceptance gate. Compare each score with the backend single-window call; preserve timestamps/order and explicitly classify missing windows.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.3 P4.03: Define normalized scores

Implementation. Implement a documented normalization, such as per-packet score $s_t / \max(N_t, 1)$, while treating silent windows according to policy rather than dividing silently.

Verification and acceptance gate. Test totals $N_t = 0, 1$, and large N_t; assert finite values or explicit non-applicability and record the formula in artifacts.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.4 P4.05: Calibrate empirical quantiles

Implementation. Compute the lower-tail threshold $\tau = Q_q(\{s_i\}_{i=1}^m)$ with declared quantile convention and target calibration false-positive rate.

Verification and acceptance gate. Compare with an independent quantile implementation, test tiny/tied samples, and verify the observed calibration alarm rate matches the convention within finite-sample limits.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.5 P4.07: Persist threshold artifacts

Implementation. Store model checksum, score type, quantile, threshold, calibration range/fingerprint, sample count, backend, and schema version.

Verification and acceptance gate. Round-trip and checksum-test the artifact. Scoring must reject a threshold created for another model, backend convention, or category schema.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.6 P4.08: Implement online scoring

Implementation. Consume windows in timestamp order and emit score records without future context. Keep state bounded and deterministic.

Verification and acceptance gate. Replay the same stream in different chunk sizes and assert identical ordered outputs; monitor memory over a long synthetic stream.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.7 P4.09: Implement anomaly decisions

Implementation. Apply the strict documented rule $s_t < \tau$, including explicit equality behavior, and retain both score and threshold.

Verification and acceptance gate. Test values below, equal to, and above threshold, including floating-point neighbors. Decisions must be backend-metadata aware.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.8 P4.11: Compute category residual explanations

Implementation. For $N > 0$, calculate

$$r_k = \frac{x_k}{N} - \frac{\alpha_k}{\alpha_0}, \qquad e_k = N\frac{\alpha_k}{\alpha_0},$$

and rank positive and negative deviations; use a separate explanation for silence.

Verification and acceptance gate. Hand-check a known vector, assert residuals sum to zero within tolerance, category ordering is deterministic under ties, and silent windows never divide by zero.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.9 P4.13: Define anomaly event schemas

Implementation. Create a versioned event containing device/window IDs, counts, score, threshold, decision, explanation, precision status, model/threshold versions, and run ID.

Verification and acceptance gate. Schema-test mandatory fields, reject incompatible category order, round-trip JSON/CSV, and verify no packet payload or unnecessary endpoint identifier is present.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.10 P0.12: Create the end-to-end smoke command

Implementation. Provide one command that validates configuration, processes the smoke fixture, trains, calibrates, scores, and writes a report.

Verification and acceptance gate. Execute from a clean environment; assert successful exit, expected artifact set, valid schemas and checksums, deterministic decisions on repeated runs, and no modification of inputs.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: One command takes the smoke fixture through preprocessing, training, LM-backed scoring, threshold calibration, and schema-valid anomaly events. This is the working-prototype milestone.

8 Deferred Until After the Working Prototype

The omitted roadmap tasks fall into the following later categories:

- **Additional engineering assurance:** broader CI, coverage, property-based testing, structured logging, container hardening, governance, threat modeling, and expanded reports.
- **Research validation:** full anomaly-injection suite, detector baselines, uncertainty estimates, sensitivity/ablation studies, locked final evaluation, and publication figures.
- **Additional functionality:** adaptive retraining, forecasting, and advisory response support.
- **Deployment and polishing:** optimized native bindings, edge benchmarks, supply-chain controls, restart/exhaustion testing, deployment documentation, and release packaging.

After this document's final gate passes, return to the Atomic Implementation Roadmap and complete the deferred tasks required by the thesis scope before making research-quality, security, or deployment-readiness claims.